

typically operate as a 'community', with people working from different locations across the globe and contributing. We look for people with capabilities to work with their remote counterparts, and that requires a different set of skills than working in a team sitting around across the cubicles. It means that people should be able to work with different cultures, understand what their counterparts are saying and also be able to communicate effectively. The candidates should be able to talk about their work openly because people don't see each other day in and day out. So they will need to know the developer as a person.

There will be appreciation and criticism at the same time. If they contribute some code which is not good, people may trash it, so they should be able to handle that kind of pressure. They don't have managers who can shield them from such things on an everyday basis. People should have the maturity to represent themselves and to communicate effectively with counterparts whom they have not met and do not know. They just know their counterparts as email addresses or by name. These things require slightly different skills. So when we hire for open source, we look at some of those skills apart from the regular programming and technical skills that we expect.

In projects with SUSE and Attachmate, we have different ways of hiring people. We have 'Bootcamps', where people can contact us and participate in a contest. The candidates are given programming challenges. Those who clear the contest can become a part of the open source project. The contests for different projects have different formats. So, hiring for open source is a slightly different ball game. The technical skills won't be drastically different but the people should have a high-level of passion for their work, should be self-driven and objective

about criticism. That, we see, as the differentiating factor. As far as the technical skills are concerned, if they have the right attitude, they can pick up technical skills easily, from project to project.

Q Do you also provide training to the people you hire?

Yes, of course. The training is very specific to the kind of projects they do and the skill sets that are required for the project. Once the candidates join us, we expect them to have some technical skills, since we do intense training. We believe in on-the-job learning, so that people can pick up on the technologies that we use.

Q We know that colleges do not provide the kind of education that makes students industry-ready and they need some extra certifications. What kind of certifications should one go for and does having them add to a professional's profile?

Certifications do make a lot of difference, especially for open source projects because they help students pick up the specific skills required for open source jobs. Last year, we tied up with some colleges to encourage certifications, and we helped them out with material. We do this on a regular basis.

Q Do you interact with the online open source community as well?

Yes, we do a lot of that. In fact, this is something very fundamental to open source development. We interact with a lot of open source communities across the globe—some of them are related to the GNOME project, the iFolder project, the KDE project and the Linux kernel project. There are many projects of which we are either the owners or the contributors. 

(Continued from Page 84....)

planning. In real-world projects, dynamic analysers are used on a 'need basis'. For example, if there are many memory leaks in a C/C++ program, projects usually try applying tools such as Rational Purify or Valgrind, which are useful. However, a better approach is to integrate such tools as part of the testing infrastructure, and run them as part of regular tests. You may be surprised to see dynamic analysers identify new bugs from the same testing infrastructure, as the development progresses! Most programmers are surprised with this simple suggestion that I've given here. They never consider using dynamic analysers regularly, and think that they are only for 'need-based' use.

A final warning: Don't blindly trust the results of program analysis tools! There have been cases in which trying to fix the warnings from static or dynamic analysers has resulted in bugs! Here is an example.

In 2008, a security vulnerability was found in Debian's OpenSSL package; the root cause was traced to a change

made two years earlier. In 2006, two lines were removed from `crypto/rand/mdrand.c` with the comment, "Don't add uninitialised data" (see Figure 1). This change was done because the Valgrind tool complained about the use of an uninitialised variable. However, the code deliberately used the uninitialised value to add 'entropy' to the random-number generator. Because of this change of not adding uninitialised data, the randomness in Debian-generated keys (SSL and SSH) was reduced to 15 bits. The only 'randomness' used was the process ID, whose value is 2^{15} , or 32,768 unique keys. The result was obviously a security vulnerability! The fix was to 'undo' the change and retain the uninitialised variable. This is a good example to understand how intentional errors are used in practice, and why we should not blindly trust the results of program analysis tools. 

By: S G Ganesh

The author works for Siemens (Corporate Research & Technologies), Bengaluru. You can reach him at sgganesh@gmail.com.